ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ РЕСПУБЛИКИ ХАКАСИЯ
«ХАКАССКИЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

ОТЧЕТ

по учебной практике
по профессиональному модулю

ПМ.01. РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

специальности 09.02.07 Информационные системы и программирование
Квалификация Программист

Студент гр. ИС(ПРО)-41 _____ Мегедь К. А.
подпись *Фамилия И.О.*

Руководитель практики от ГБПОУ РХ ХПК _____ _____ _____ Черкашин Д.С.
оценка *дата* *подпись* *Фамилия И.О.*

Абакан 2026 г.

Содержание

1 Тестирование программных модулей.....3
1.1 Разработка тест-кейсов 3
1.2 Модульное тестирование.....7

1 Тестирование программных модулей

Тестирование — это процесс проверки продукта на соответствие требованиям и поиска ошибок до того, как им начнут пользоваться люди. Его проводят, чтобы найти дефекты как можно раньше, когда их исправление обходится дешевле, убедиться, что система работает именно так, как задумано, снизить риски сбоев и потери данных в продакшене, а также повысить общее качество и надёжность продукта.

1.1 Разработка тест-кейсов

Тест-кейс — это документированный набор шагов, условий и ожидаемых результатов для проверки конкретной функции или требования. Он нужен для стандартизации процесса тестирования, обеспечения полноты покрытия и возможности повторного запуска проверок. Основные атрибуты: уникальный ID, название, описание, предусловия, шаги выполнения, ожидаемый результат, приоритет и статус. Чёткая структура позволяет любому тестировщику воспроизвести проверку и однозначно интерпретировать итог. Тест-кейсы также служат живой документацией поведения системы и основой для отчётности о качестве продукта.

Разработка тест-кейсов.

Таблица 1 — Тест-кейс №1.

Номер	1
Название	Проверка запуска приложения.
Цель	Проверить работоспособность приложения путем его запуска
Предусловие	Нет
Шаги	1. Запуск приложения. (.venv) [student-pro@b206-3 demkaproject]$ python manage.py runserver Watching for file changes with StatReloader Performing system checks... System check identified no issues (0 silenced). February 24, 2026 - 05:28:25 Django version 4.2.28, using settings 'demkaproject.settings' Starting development server at http://127.0.0.1:8000/ Quit the server with CONTROL-C. 2. Открытие приложения. Добро пожаловать, гость! Авторизоваться
Входные данные	Нет

Ожидаемый результат	Запуск производится успешно, страница открывается
Статус	Выполнено

Таблица 2 — Тест-кейс №2.

Номер	2
Название	Проверка авторизации
Цель	Проверка работоспособности авторизации
Предусловие	Присутствую данные для авторизации.
Шаги	1. Запуск приложения. (.venv) [student-pro@b206-3 demkaproject]$ python manage.py runserver Watching for file changes with StatReloader Performing system checks... System check identified no issues (0 silenced). February 24, 2026 - 05:28:25 Django version 4.2.28, using settings 'demkaproject.settings' Starting development server at http://127.0.0.1:8000/ Quit the server with CONTROL-C. 2. Переход на форму для авторизации. **Вход в систему** Логин: Пароль: Войти 3. Ввод логина и пароля. **Вход в систему** Логин: 94d5ous@gmail.cor Пароль: •••••• Войти 4. Авторизация и отображение авторизованного пользователя. **Панель Администратора** Добро пожаловать, **Никифорова Весения Николаевна**! Роль: Администратор **Выйти**
Входные данные	Логин и пароль
Ожидаемый результат	Пользователь авторизован и выводятся его данные
Статус	Выполнен

Таблица 3 — Тест-кейс № 3.

Номер	3
Название	Проверка отображения данных
Цель	Проверка функциональности отображения данных
Предусловие	Нет
Шаги	1. Запуск приложения. (.venv) [student-pro@b206-3 demkaproject]$ python manage.py runserver Watching for file changes with StatReloader Performing system checks... System check identified no issues (0 silenced). February 24, 2026 - 05:28:25 Django version 4.2.28, using settings 'demkaproject.settings' Starting development server at http://127.0.0.1:8000/ Quit the server with CONTROL-C. 2. Отображение данных. Добро пожаловать, гость! Авторизоваться Категория: Женская обувь | Название: Ботинки Описание: Ботинки Marco Tozzi женские демисезонные, размер 39, цвет бежевый Производитель: Marco Tozzi Поставщик: Обувь для вас Цена: ~~3244.0 тенге~~ 3179 тенге Единица измерения: шт. Количество на складе: 13
Входные данные	Нет
Ожидаемый результат	Отображение карточек с выведенными данными
Статус	Выполнено

Таблица 4 — Тест-кейс №4.

Номер	4
Название	Проверка отображения скидки и её функций
Цель	Проверка функционирования отображения скидки, и скидки превышающей 15%
Предусловие	Данные отображаются, у каждой карточки своя скидка
Шаги	1. Запуск приложения.

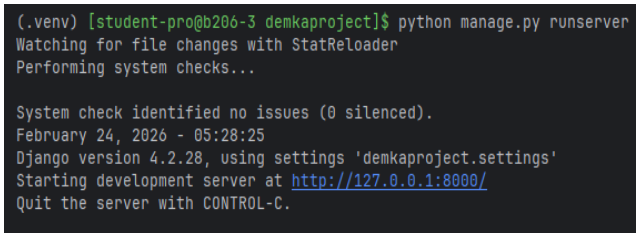
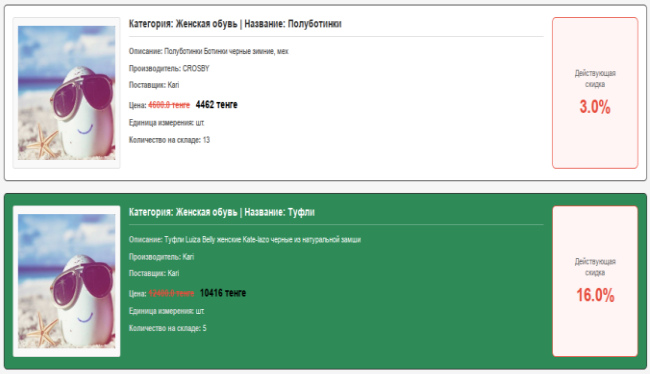
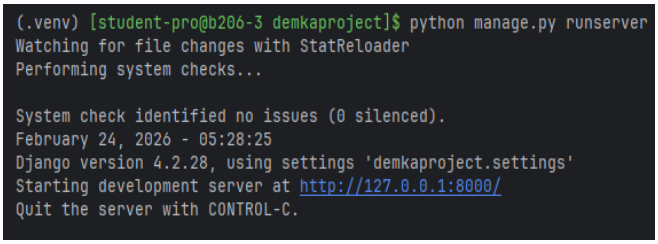
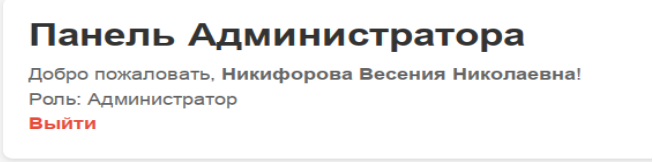
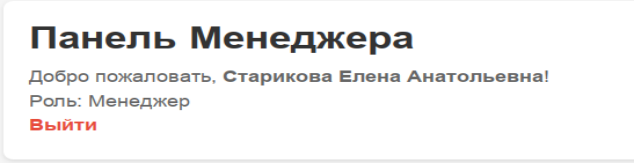
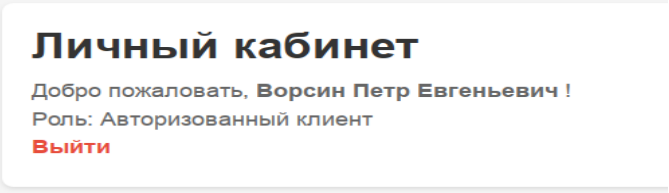
	 2. Отображение данных. 3. Проверка отображения скидки и её функции. 
Входные данные	Нет
Ожидаемый результат	Скидка отображается как и остальные данные, если скидка больше 15% карточка подсвечивается зеленым
Статус	Выполнено

Таблица 5 — Тест-кейс №5.

Номер	5
Название	Проверка на разделение пользователей
Цель	Проверка функции авторизации пользователей при разных входных данных и отображение информации о пользователе
Предусловие	Нет
Шаги	1. Запуск приложения.

	 2. Авторизация от имени администратора.  3. Авторизация от имени менеджера.  4. Авторизация от имени авторизованного клиента. 
Входные данные	Логин и пароль
Ожидаемый результат	Разделение пользователей производится успешно
Статус	Выполнено

1.2 Модульное тестирование

Модульное тестирование — это проверка отдельных единиц кода, например функций, в изоляции. Оно нужно для раннего поиска ошибок, упрощения рефакторинга и гарантии корректности логики. Структура теста обычно строится по принципу AAA: подготовка данных и окружения (Arrange), вызов тестируемого метода (Act) и проверка результата через утверждения (Assert). Это позволяет разработчикам быстро локализовать дефекты и поддерживать высокое качество продукта. Важно, чтобы каждый тест был независимым, быстрым в выполнении и легко читаемым для команды.

Выполнение модульного тестирования.

Листинг 1 - Импорт данных

```
from django.test import TestCase, Client  # Импортируем базовый класс для тестов и клиент для эмуляции запросов
from .models import UserEmployes, RoleEmployes, Product  # Импортируем модели, которые будем использовать в тестах
```

```
from django.urls import reverse  # Импортируем функцию для получения URL по имени представления
```

Листинг 2 - Объявление класса для тестов.

```
class ModuleTests(TestCase):    # Объявляем класс тестов, наследуясь от TestCase Django
    def setUp(self):   # Метод setUp выполняется перед каждым тестом для подготовки данных
        self.client = Client()  # Инициализируем тестовый клиент для отправки HTTP-запросов
        # Создаем роль для тестового пользователя
        self.role = RoleEmployes.objects.create(name='Администратор')
        # Создаем тестового сотрудника с логином и паролем
        self.user = UserEmployes.objects.create(
            roleemployees=self.role,
            FIO='Иванов И.И.',
            login='admin',
            password='12345'
        )
```

Описание: Проверяет корректность обработки POST-запроса с валидными данными, установку сессионных переменных и редирект на страницу администратора.

Листинг 3 - Тест 1: Проверка успешной авторизации пользователя

```
def test_login_view_success(self):
 data = {'username': 'admin', 'password': '12345'}   # Формируем словарь с данными для входа (логин и пароль)
        response = self.client.post('/login/', data)   # Отправляем POST-запрос на URL авторизации с данными
        self.assertEqual(response.status_code, 302)  # Проверяем, что статус ответа 302 (редирект после успеха)
        self.assertEqual(response.url, '/admin/')  # Проверяем, что редирект ведет на страницу администратора
        self.assertEqual(self.client.session['user_id'],
                         self.user.id)   # Проверяем, что ID пользователя записан в сессию
        self.assertEqual(self.client.session['user_fio'], 'Иванов И.И.')   # Проверяем, что ФИО записано в сессию
```

Описание: Проверяет корректность преобразования строковых полей цены и скидки в числа и вычисления итоговой стоимости товара в контексте ответа.

Листинг 4 - Тест 2: Проверка логики расчета цены и скидки в представлении index.

```
def test_index_view_price_calculation(self):
  # Создаем тестовый товар с ценой 1000 и скидкой 20% (строки, как в модели)
        product = Product.objects.create(
            article='ART001',
            price='1000',
            discount='20',
            quantity='10',
            description='Test'
        )
        response = self.client.get('/')  # Отправляем GET-запрос на главную страницу (index)
        self.assertEqual(response.status_code, 200)   # Проверяем успешный статус ответа (200 OK)
        context = response.context['tovars'][0]  # Получаем первый элемент из списка товаров в контексте
        # Проверяем, что итоговая цена рассчитана верно: 1000 * (1 - 20/100) = 800.0
        self.assertEqual(context['final_price'], 800.0)
```

```
        self.assertTrue(context['has_discount'])     #  Проверяем,  что  флаг
наличия скидки установлен в True
        self.assertFalse(context['out_of_stock'])     #  Проверяем,  что  флаг
отсутствия на складе установлен в False
```

Описание: Проверяет, что при отсутствии сессии (пользователь не вошел) доступ к странице администратора запрещен и происходит редирект на логин.

Листинг 5 - Тест 3: Проверка защиты страницы администратора

```
def test_admin_page_access_denied(self):
 response  =  self.client.get('/admin/')    #  Отправляем  GET-запрос  на
страницу администратора без авторизации
             self.assertEqual(response.status_code,  302)   #  Проверяем,  что
сервер возвращает код редиректа
             self.assertEqual(response.url,  '/login/')    #  Проверяем,  что
редирект ведет на страницу входа
             # Убеждаемся, что в сессии нет ключа user_id (доступ закрыт)
             self.assertNotIn('user_id', self.client.session)
```

Описание: Проверяет, что запрос на выход очищает сессионные данные и перенаправляет пользователя на страницу авторизации.

Листинг 6 - Тест 4: Проверка функционала выхода из системы

```
def test_logout_view(self):
# Сначала авторизуемся, чтобы создать сессию
             self.client.post('/login/',  {'username':  'admin',  'password':
'12345'})
             self.assertIn('user_id', self.client.session)  # Убеждаемся, что
сессия активна перед выходом
             response = self.client.get('/logout/')  # Отправляем GET-запрос
на URL выхода
             self.assertEqual(response.status_code, 302)  # Проверяем статус
редиректа
             self.assertEqual(response.url, '/login/')  # Проверяем редирект
на страницу входа
             # Проверяем, что сессия очищена (ключа user_id больше нет)
             self.assertNotIn('user_id', self.client.session)
```

Приложение А.

Программный код в системе контроля версий

Ссылка: https://github.com/az7fd/-_-01.02_-